

• Software Architecture CoP - Architecture KATAS Edition #2



- Kamal Mehta, Serhii Ziniuk, Shivani Muthu
- Mar 2026

Road Warrior

A major travel agency wants to build the next generation online **trip management dashboard** to allow travelers to see all their **existing reservations** organized by trip either online or through their mobile device.

- **Users:**

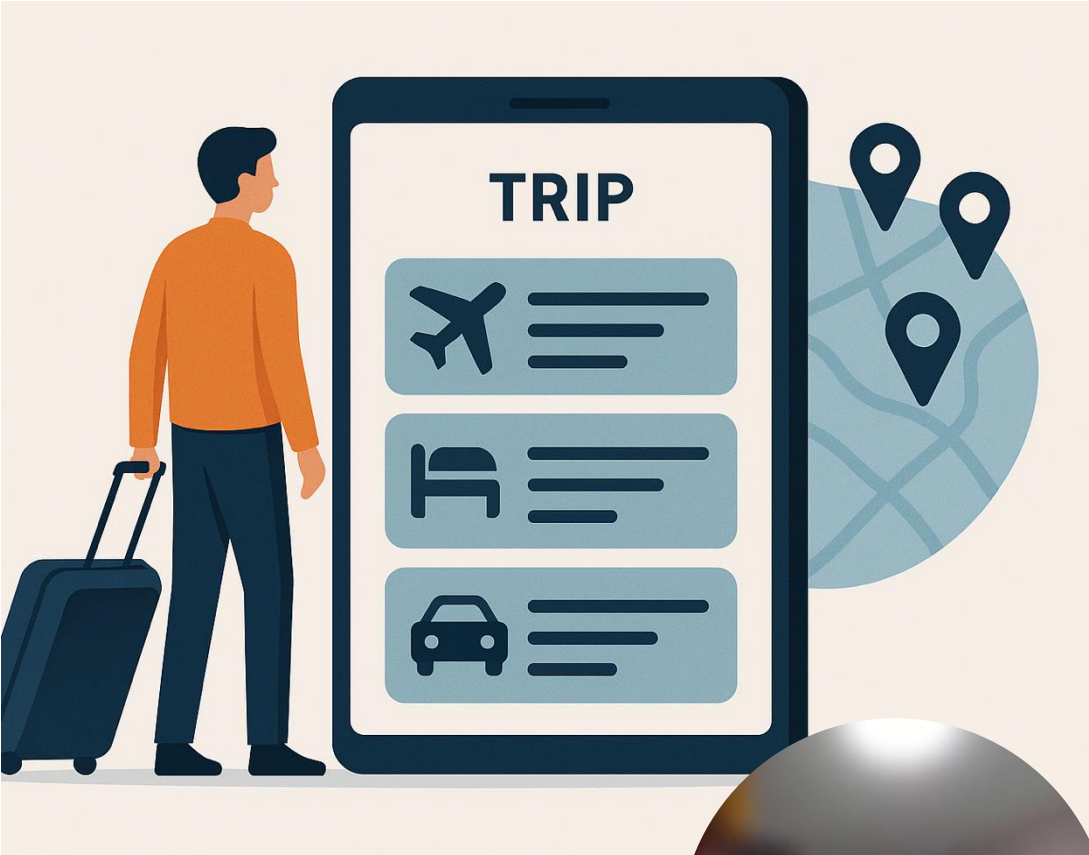
- The system will initially support 10,000+ registered travelers **worldwide**
- Expected growth trajectory toward 50,000–100,000 active users over the next 2–3 years.
- Peak usage is expected during major travel seasons, where concurrent usage may reach several thousand users across web and mobile platforms.

- **Requirements:**

- The system must connect with **the agency’s existing airline, hotel, and car rental interface system** to automatically load reservations via frequent flier accounts, hotel point accounts, and car rental rewards accounts.
- Customers should be able to **add existing reservations manually** as well.
- Items in the dashboard should be able to be grouped by trip, and once the trip is complete, the items should automatically be removed from the dashboard and archived for example "Past Trips".
- Users should also be able to share their trip information by interfacing with standard social media sites.
- Richest user interface possible across all deployment platforms.

- **Additional Context:**

- Must integrate seamlessly with existing travel systems; partnership deals are being negotiated to create 'favored' vendors; must work internationally



- KATAS

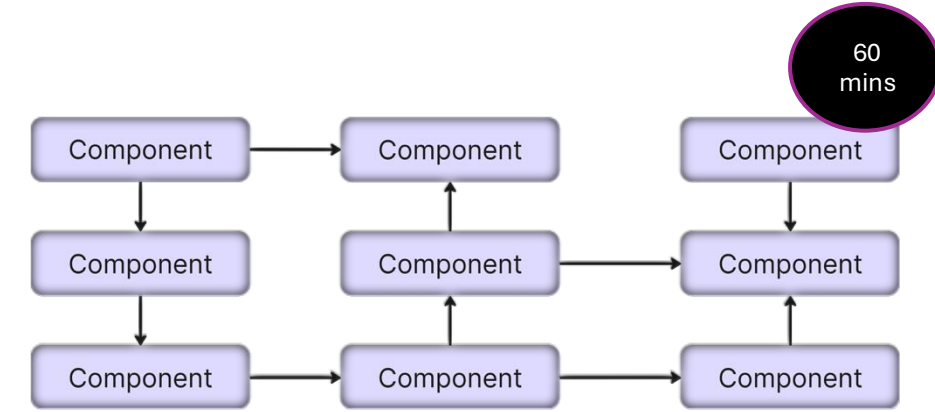
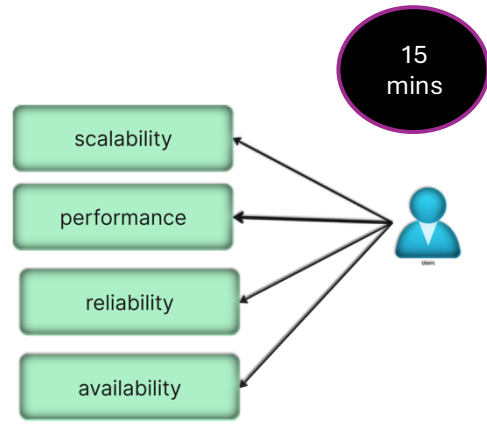
The process

Your Architectural Kata is...
Going Going Gone!

An auction company wants to take their auctions online to a nationwide scale—customers choose the auction to participate in, wait until the auction begins, then bid during the live auction as if they were there in the room, with the auctioneer.

- Users:** scale up to hundreds of participants (per auction), potentially up to thousands of participants, and as many simultaneous auctions as possible
- Requirements:**
 - bidders can see a live video stream of the auction and see all bids as they occur
 - auctions must be as real-time as possible
 - both online and live bids must be received in the order in which they are placed
 - bidders register with credit card; system automatically charges card if bidder wins
 - participants must be tracked via a reputation index
- Additional Context:**
 - auction company is expanding aggressively by merging with smaller competitors
 - if nationwide auction is a success, replicate the model overseas
 - budget is not constrained—this is a strategic direction
 - company just exited a lawsuit where they settled a suit alleging fraud

10
mins



1 Identify Purpose, Goals & Motivation

2 Identify Architecture Characteristics

3 Component Identification - Logical Architecture



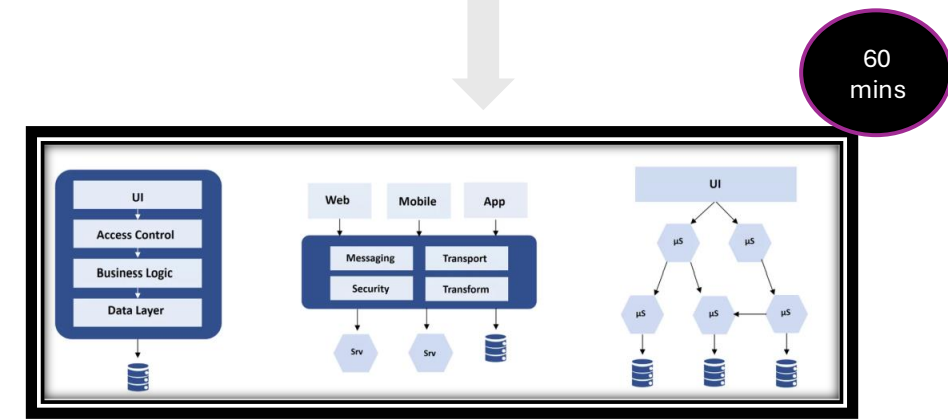
Architecture Decision Record (ADR) Template

Title	Short name of the decision to be made
Date	Date when the decision was made
Context	Explanation of the problem
Decision	Clear statement of the decision made
Alternatives	Description of alternative solutions that were considered
Consequences	Implications of the decision, positive or negative
Participants	Names of team members how made the decision
Status	The current status of the decision, such as, "proposed," "accepted," "rejected," or "superseded."

ACD

1. Introduction & Goals Fundamental requirements, esp. quality goals	7. Deployment View Hardware, infrastructure & deployment
2. Constraints Regulations and external constraints	8. Crosscutting Concepts Cross-cutting topics, often very technical and detailed
3. Context & Scope External systems & interfaces	9. Architectural Decisions Specific architectural decisions (not described elsewhere)
4. Solution Strategy Core ideas and solution approaches	10. Quality Requirements Quality tree, quality scenarios
5. Building Block View Structure of source code, modularization (hierarchical)	11. Risks & Technical Debt Known problems and risks
6. Runtime View Important runtime scenarios	12. Glossary Important and specific terms ("ubiquitous language")

35
mins



5 Architecture Documentation

4 Identify Architecture Style and build C4 Models

What is expected

✓ ACD Checklist

- ☐ Functional requirements & Non-Functional requirements (***Architectural Characteristics sheet*** – Section 1 in ACD)
- ☐ Logical architecture (***C4 Container Diagram*** – Section 5.1 in ACD)
- ☐ Physical architecture (***Deployment Diagram*** – Section 7 in ACD)
- ☐ Solution strategy (Map requirements to your decision – Section 4 in ACD)
- ☐ List the **ADRs** in a very short format (optional)

📌 Check-In & Code Submission Expectations

- ☐ You will be provided a repo where you can check in your code, by your SPOC
- ☐ Make sure you commit your changes **by the EOD (EST) you selected.**
- ☐ The revision that gets evaluated will be based on the **EOD check-in.**



Participating teams

Team 1	Team 2 (IST)	Team 3 (EST)	Team 4(IST)	Team 5
Kaivalya Mete	Aditi Patil	Khalid Bashir	Karthikeyan Vedi	Yuvraj Dhakar
Suraj Vishwakarma	Prathamesh Jadhav	Ekin Ozcelik	Subrato Pattanaik	Abhijit Kulkarni
Monika Gonnade	Yash Ghorpade	Toufiq Hossain	Afraj Attar	Aditi Hajare
Ankit Dhume	Aditya Kavale	Matthew Spring	Pranjal Kumar	Aditya Patil
	Sushmita Sathe	Matthew Nitsopoulos	Ajinkya Gholape	Harshal Shirke
			Shivam Chaurasiya	Saniya Gupte
Date: March 6th	Date: March 9th	Date: March 4th	Date: March 5th	Date: March 6th
SPOC: Serhii Zinuik	SPOC: Shivani Muthu	SPOC: Kamal Mehta	SPOC: Serhii Zinuik	SPOC: Shivani Muthu

Worksheet to evaluate Architecture Characteristics

Architecture Characteristics Worksheet

System/Project: _____

Domain/Quantum: _____

Architect/Team: _____

Date: _____ Next Review: _____

Top 3 Driving Characteristics

☐

☐

☐

☐

☐

☐

☐

Implicit Characteristics

feasibility (cost/time)

security

maintainability

observability

Others Considered

Instructions

• Identify no more than 7 driving characteristics.

• Pick the top 3 characteristics (in any order).

• Implicit characteristics can become driving characteristics if they are *critical* concerns.

• Add additional characteristics identified that weren't deemed as important as the list of 7 to the *Others Considered* list.

• Definitions are on the following pages

Common Architecture Characteristics

performance

data integrity

deployability

responsiveness

data consistency

testability

availability

adaptability

abstraction

fault tolerance

extensibility

workflow

scalability

interoperability

configurability

elasticity

concurrency

recoverability

Composite Architecture Characteristics

agility → maintainability, testability, deployability

reliability → availability, testability, data integrity, data consistency, fault tolerance

a

b

denotes characteristics that are related; some systems only need one of these, other systems may need both

Last updated March 2024

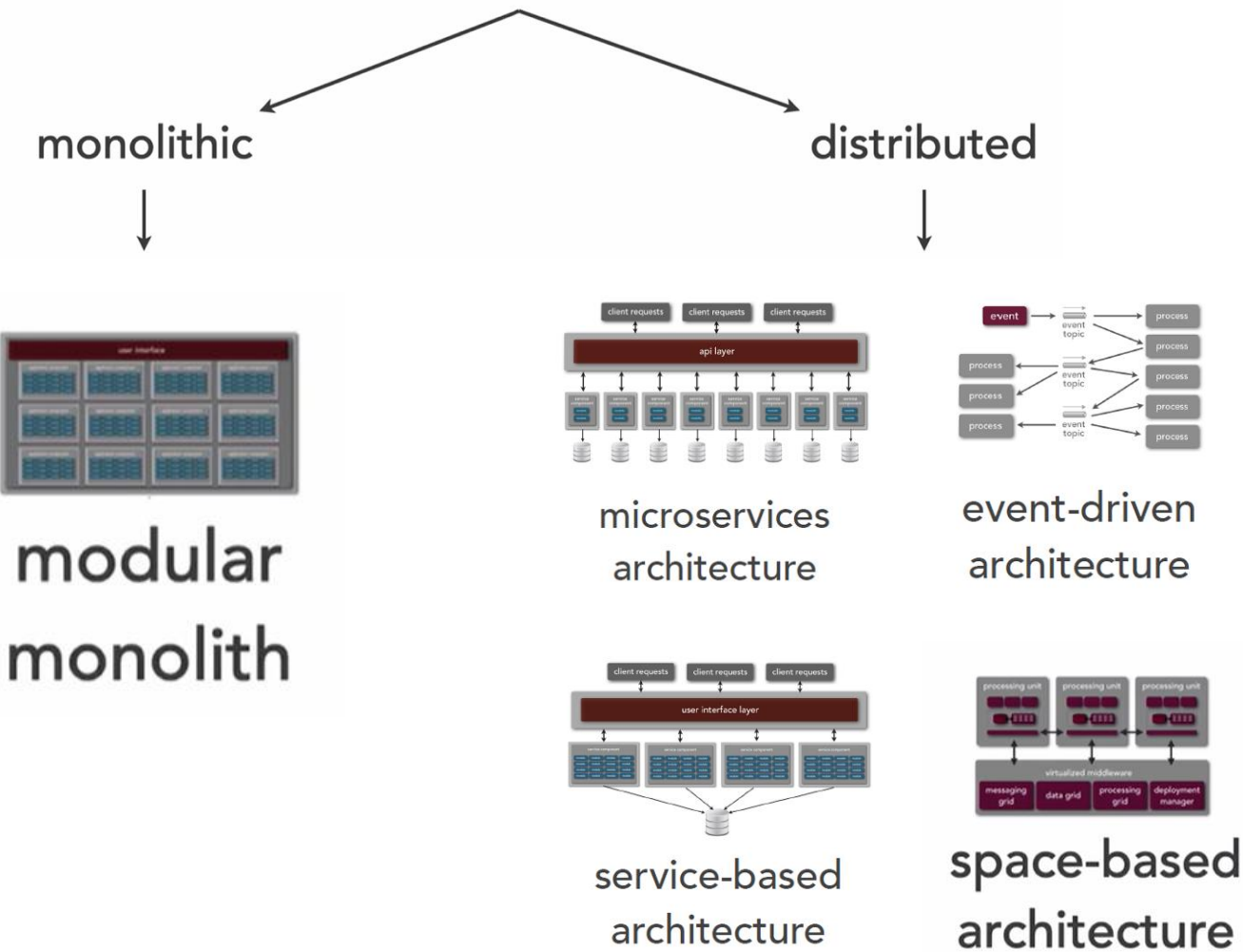
Created by Mark Richards, DeveloperToArchitect.com

Source: <https://www.developertoarchitect.com/resources.html>

[Click here to access your Architecture Characteristics worksheet](#)

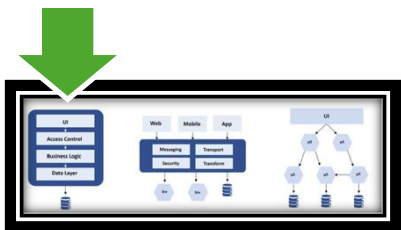
Identify Architecture Styles

architecture styles



Architecture styles help define the basic characteristics and behavior of an application

Two or more styles can co-exist in a system



Architecture Styles with Rating

Characteristic	Microservices	Service-Based	Event-Driven	Space-based
Partitioning	Domain	Domain	Technical	Technical
Cost	\$ \$ \$ \$	\$ \$	\$ \$ \$	\$ \$ \$ \$
Maintainability	★★★★★	★★★★★	★★★★☆	★★★☆☆
Testability	★★★★★	★★★★★	★★★☆☆	★★★☆☆
Deployability	★★★★★	★★★★★	★★★★☆	★★★★☆
Simplicity	★★★☆☆	★★★★☆	★★★☆☆	★★★☆☆
Scalability	★★★★★	★★★★☆	★★★★★	★★★★★
Elasticity	★★★★☆	★★★☆☆	★★★★☆	★★★★★
Responsiveness	★★★★☆	★★★★☆	★★★★★	★★★★★
Fault-Tolerance	★★★★★	★★★★☆	★★★★★	★★★★☆
Evolvability	★★★★★	★★★★☆	★★★★★	★★★★☆
Abstraction	★★★☆☆	★★★☆☆	★★★★☆	★★★☆☆
Interoperability	★★★★☆	★★★☆☆	★★★★☆	★★★☆☆
Performance	★★★☆☆	★★★★☆	★★★★★	★★★★★

[Click here to access your Architecture Styles worksheet](#)

References:

- Click here to download
 - [Arc42 template](#)
 - [Architecture Styles Worksheet](#)
 - [Architecture Characteristics Worksheet](#)
 - [ADR template](#)
 - [More info – Kata pre-read](#)
 - [C4 Container Diagram](#)
 - [C4 Deployment Diagram](#)
 - [Puml online editor](#)
 - [C4 model puml lib](#)
- Video lessons:
 - [Arch Kata explanation](#)
 - [Logical architecture components](#)
 - [Logical components: workflow approach](#)
 - [Logical components: Actor/Action approach](#)
 - [Modularity and Architectural Styles](#)
 - [Logical vs Physical architecture](#)

1. Core Scope of the Kata

Assumption

The system does not perform any booking operations.

The application only consumes and displays reservation data that already exists in external airline, hotel, and car -rental systems via the agency interface. Any workflows related to searching, pricing, payment, or creating reservations are explicitly *out of scope*.

2. Systems the Solution Integrates With

Assumption

The system integrates exclusively with the agency’s existing interface layer, which aggregates all airline, hotel, and car-rental providers.

The solution has **no direct integrations with external vendors**, and it does not support onboarding, configuring, or maintaining vendors.

Any new vendors added in the future will be handled by the agency interface without requiring changes in our system.

3. How Reservations Are Loaded and Updated

Assumption

The agency interface is responsible for providing reservation data and keeping it fresh.

The system receives updated reservations either via **event-driven updates** or **periodic polling**, depending on the team’s chosen design.

The team will specify an assumed update cadence (e.g., every 5 minutes, or event messages as they occur).

The dashboard is expected to reflect changes such as delays, cancellations, or rebookings.

4. Manual Reservation Entry

Assumption

Manual entry is a fallback workflow used only when automated loading from the agency system fails or is unavailable.

Users will manually input a reservation using the **same data structure** the agency system would normally return (dates, vendor name, locations, times, etc.).

The specific UI pattern (full form vs. entering a confirmation code/PNR) is at the team’s discretion and will be documented.

5. Automatic vs. Manual Trip Grouping

Assumption

The system will automatically group reservations into trips based on date ranges and logical travel patterns.

Because the problem statement leaves room for interpretation, teams may additionally allow **user-controlled overrides** (e.g., manual re-grouping or defining custom trips), but this is optional.

All grouping logic assumptions will be explicitly documented.

FAQs

6. Completed Trips

Assumption

The system automatically archives trips after their end date.

Once a trip has concluded, all associated reservations are removed from the active dashboard and moved into a “Past Trips” or equivalent archived section. No additional workflow (e.g., approvals, cleanup, or user confirmation) is required unless the team chooses to add it as an enhancement.

7. Social Media Sharing

Assumption

Trip sharing is limited to simple, text-based posts.

The system will generate a short textual summary of a trip (e.g., “Traveling from City A to City B”) that users can share to standard social media platforms. More advanced sharing mechanisms—such as deep links, live status sharing, or authenticated integrations—are considered optional and outside baseline scope.

8. Notifications and Real-Time Updates

Assumption

Reservation updates are provided automatically by the agency systems.

Teams may assume either event-driven updates or periodic polling to keep trip details current. Notifications (e.g., delays, cancellations, gate changes) may be included for improved user experience, provided the team documents the chosen update and notification mechanism.

9. “Richest User Interface Possible”

Assumption

The phrase encourages thoughtful UX, not full design mockups.

Teams are not required to deliver detailed UI wireframes or pixel-perfect layouts. Instead, they should emphasize UX principles—such as organization, clarity, and cross-platform usability—and connect those choices back to their architecture and API design.

10. Favored Vendors

Assumption

Favored vendors are determined by the agency systems, not by this application.

The system only needs to visually indicate favored vendors (e.g., a badge or highlight) when displaying reservations. No preference logic, ranking algorithms, or booking prioritization is required since booking workflows are out of scope.

11. Architectural Placement

Assumption

The system operates within the agency’s trusted environment unless stated otherwise.

Teams should assume a simple, internal deployment model where all integrated systems coexist within the same security boundary. If a team chooses an externalized or distributed deployment, it must document the added security considerations and integration mechanisms.

FAQs